

# **Lab 2: Heisrapport**

Eirik Stokker Aksdal, Vikingur Sigurðsson

Gruppe 11

17. mars 2026

## Innledning

Denne rapporten dokumenterer arbeidet med heisprosjektet tilhørende lab 2 i TTK4235 våren 2026. Prosjektets formål har vært å utvikle et styresystem i C for en fysisk heismodell på sanntidslaben, med fokus på strukturert programvareutvikling, modulær oppbygning og etterprøvable funksjonalitet i henhold til gitte kravspesifikasjoner.

I rapporten skal vi gå gjennom fremgangsmåten for utviklingen av programmet med utgangspunkt i V-modellen, samt programstruktur ved bruk av UML og til slutt en refleksjon av arbeidet rundt V-modellen og bruken av UML

## Fremgangsmåte

Vi startet med en prototypefase der vi skrev noenlunde fungerende kode for å forstå det utleverte rammeverket og driveren til heisen. Videre brukte vi V-modellen og erfaringene ifra prototypingen for å strukturere, planlegge og implementere den faktiske produksjonskoden.

Fra prototypefasen fant vi blandt annet at den utleverte driverkoden var ekstremt treg og til tider gjorde at heisen kjørte forbi en etasje før den registrerte den. Derfor implementerte vi Utskrift 1 i `elevio_init` funksjonen til `elevio.c` for å forbedre responsen på driveren.

```
int flag = 1;
setsockopt(
    sockfd,
    IPPROTO_TCP,
    TCP_NODELAY,
    &flag,
    sizeof(flag)
);
```

Utskrift 1: Setter et flag i initialisering som gjør at TCP sender packets individuelt istedenfor å vente på flere packets.

Siden driveren kommuniserer med elevator serveren gjennom mange små packets over TCP så ville TCP automatisk vente på flere packets

før den sender dem som en stor packet. Til vanlig er dette fornuftig siden packetsene vi sendte var ofte mindre enn overheadet som trengtes til TCP.

Etter prototypingen begynte vi med den pragmatiske V-modellen og per den så starter vi med kravspesifikasjonene til heisen der vi tar utgangspunkt i FAT-kravene gitt i oppgavebeskrivelsen og use-case diagrammet i Figur 1.

Videre gjorde vi idemyldring for arkitekturdesign gjennom å vurdere kodene skrevet i prototypefasen der vi til slutt kom fram til hvordan bestillingssystemet og arkitekturen kunne mest fornuftig implementeres. Det var feks her at vi kom fram til at vi skulle bruke en tilstandsmaskin for sluttimplementasjonen og hvilke tilstander vi kom til å trenge som er illustrert i Figur 2.

Gitt tilstandsmaskinen kan use-case diagrammet beskrives slik:

### Velg Etasje

Pre-conditions:

Heisen er initialisert og stoppknappen er sluppet.

Trigger:

Bestillingsknapp inne i heisen blir trykket.

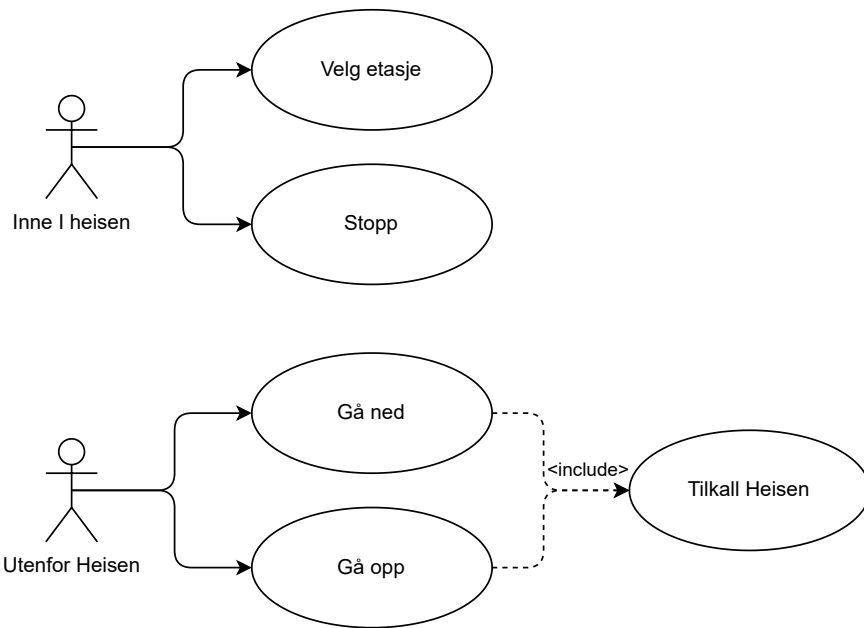
Suksess-scenario:

1. Heisens tilstand og motor settes til tilhørende retting.
2. Heisen ankommer etasjen.
3. Heisen stopper.
4. Heisen åpner døra i 3 sekunder.

Utvidelser:

1a. Hvis stoppknappen trykkes følger suksess-scenario alternativ flow ifølge use-case til stoppknappen.

2a. Døra forblir åpen dersom det er en obstruksjon



Figur 1: Use-case diagram

## Stopp

Pre-conditions:

Heisen er initialisert.

Trigger:

Stoppknappen trykkes.

Suksess-scenario:

1. Heisen stopper
2. Alle bestillinger slettes.
3. Døra åpnes dersom heisen er i en etasje

Utvidelser:

- 1a. Døra forblir åpen intill 3 sekunder etter stoppknappen trykkes
- 2a. Døra forblir åpen dersom det er en obstruksjon.

## Gå opp/ned

Pre-conditions:

Samme som Velg etasje

Trigger:

Knapp opp eller ned utenfor heisen blir trykket

Suksess-scenario:

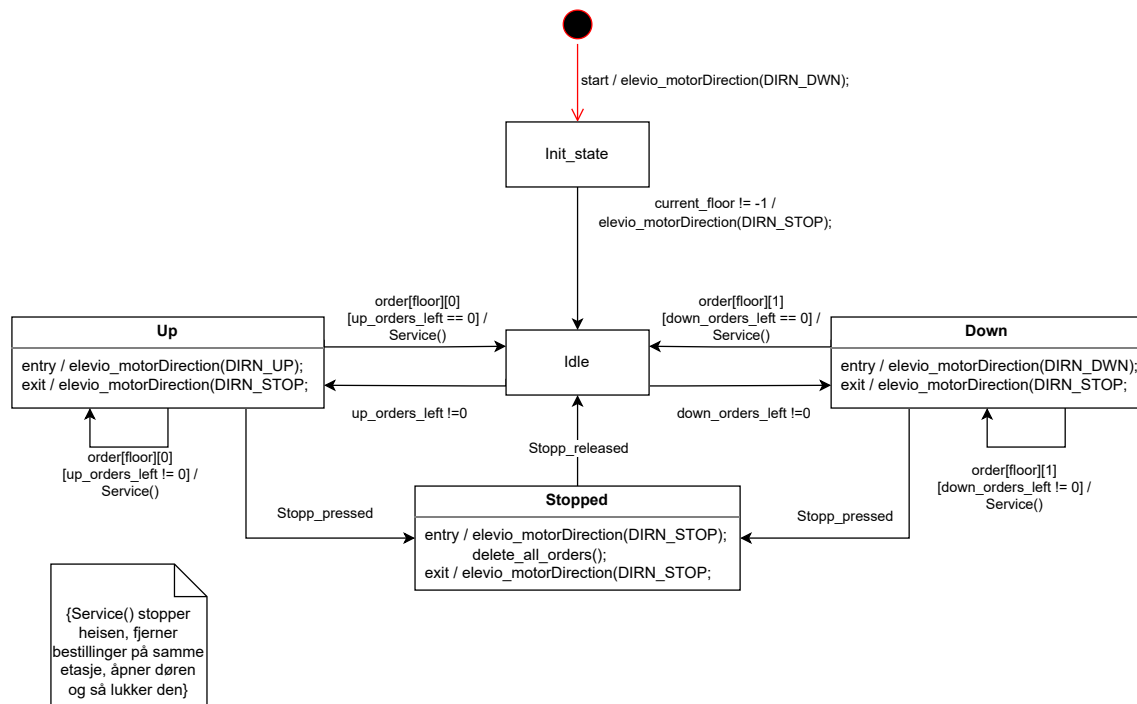
Samme som Velg etasje

Utvidelser:

Samme som Velg etasje

Når det gjelder heisens oppførsel bestemte vi at hvis den beveger seg i en retning skal den fullføre alle ordre som er forran den i den retningen, med prioritet av ordre i samme retning (dvs at den ignorerer ordre i motsatt retning hvis ordre i samme retning er tilgjengelig forran den). Og at den går i idle dersom den ikke har flere ordre i den retningen, og at idle tilstand alltid ser etter nye ordre og går i retningen til den første ordren den registrerer.

Med utgangspunkt i det oppsatte rammeverket vårt, oversatte vi og implementerte vi koden vår i C.



Figur 2: Førsteutkast av tilstands diagram

Koden vi endte opp med var delt i `main.c` og `elevatorutils.c` der `main.c` inneholder tilstandsmaskinen og `elevatorutils.c` inkluderer alle hjelpefunksjoner. Som for eksempel Utskrift 5 som sjekker om det er ordre i etasjene den er i, og fullfører den dersom den er gyldig, gitt kravene. `elevatorutils.c` kan også videre deles inn i IO funksjoner som håndterer inputs og outputs og tilstandsmaskin funksjoner som håndterer logikken i tilstandsmaskinen.

Førsteutkastet av denne implementasjonen ga en nogenlunde fungerende kode. Hver implementerte funksjon ble testet og fungerte. Men den passerte ikke alle kravene til FAT testen. Spesifikt test S6 der Stopp tilstanden kunne huske dens forrige etasje, men på grunn av tilstands oppsettet, er den også avhengig av hvilken retningstilstand den var i. Derfor endte vi opp med å legge til en til tilstand som beskriver når heisen er blitt stoppet mellom to etasjer kalt `STOP_IDLE`.

## Struktur

Testing og opprydding av kode slik at den fungerer bedre og ser finere ut, i henhold til V-modellens prosess, ga det ferdige produktet beskrevet her.

Et overblikk over programmets funksjon kan best gis med det ferdige tilstandsdiagrammet i Figur 3 og kalssedigrammet i Figur 4. Programmets funksjon kan beskrives som følger.

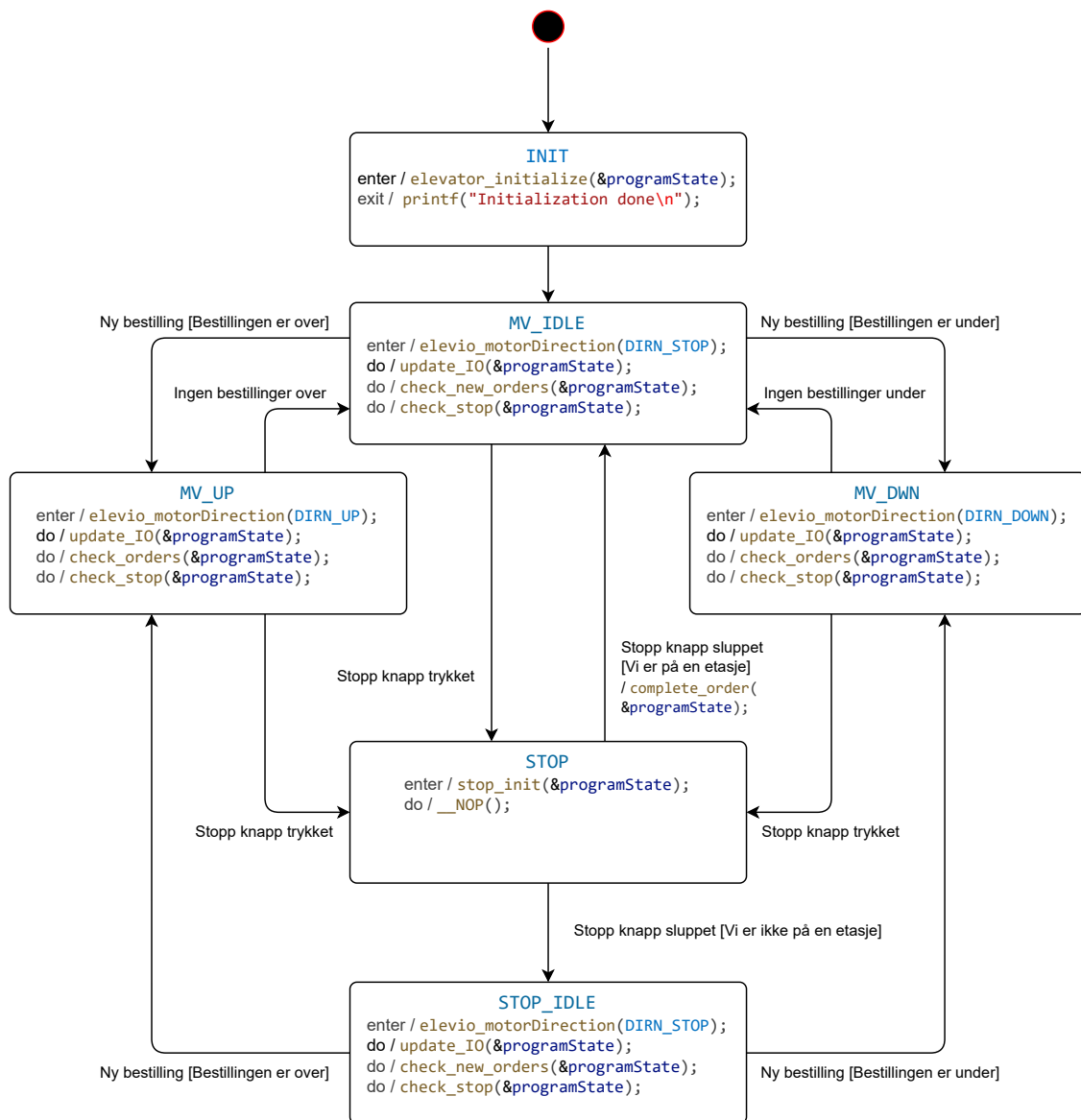
## Fellestrekk

Alle tilstander utenom stopp og init kaller Utskrift 6, som oppdaterer indikator lys og ordre basert på inputs og sensorer. Og Utskrift 7 som endrer tilstand til Stopp dersom stoppknappen blir trykket.

Heisens tilstand beskrives med enumen Utskrift 2 og structen Utskrift 3

## Initialiserings tilstand

Når programmet startes så sjekker heisen først etter gyldig etasje. Her tar den ingen inputs utenom etasje sensorene. Dersom den starter i en gyldig etasje går den umiddelbart til idle state. Hvis ikke så begynner heisen å bevege seg



Figur 3: Fullført tilstandsdiagram

nedover intill den treffer en gyldig etasje. Initialiseringen kan videre beskrives med Figur 5

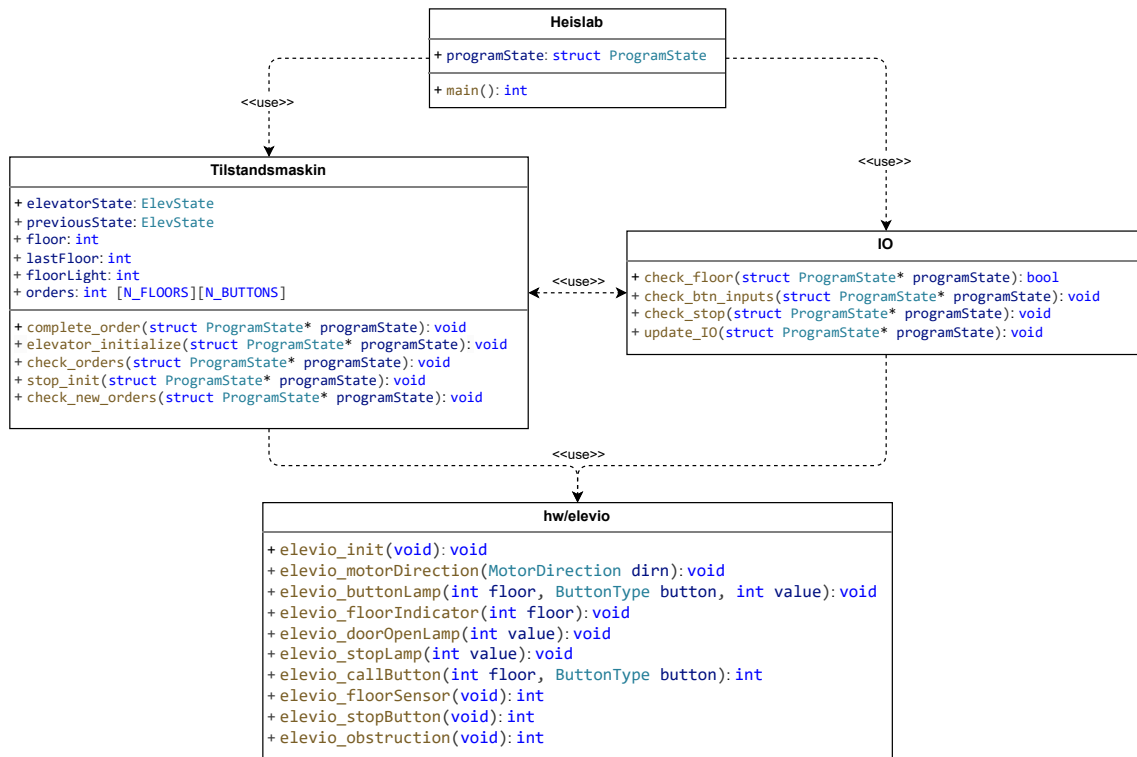
### Idle tilstand

I idle tilstand vil programmet lese ordre listen intill den finner en ny ordre. Dersom ordren er i etasjen den er i åpner den bare døren. Ellers vil den endre tilstanden sin til retningen ordren kom fra. Idle tilstanden kan videre beskrives med Figur 6

### Rettnings tilstand

Dersom heisen er i en rettnings tilstand, altså opp eller ned, så vil motoren være i den

korresponderende retningen. Dersom den er i en gyldig etasje vil den sjekke etter ordre. Først sjekker den alle ordre i retningen den går i. Dersom det er en ordre for samme retning den går i på etasjen den er i kaller den Utskrift 4 og deretter Utskrift 5 som sjekker etter flere ordre i bevegelsesretning og endrer tilstand til idle dersom det ikke er noen. Figur 7 gir videre beskrivelse av betjeningen av en bestilling over heisen. Merk at betjeningen av en bestilling under heisen er for alle praktiske formål identisk til en bestilling over.



Figur 4: Klasse diagram

## Stopp tilstand

Stopp tilstanden begynner med å slette alle ordre og skru av indikatorlys til tilhørende knapper og stopper motoren. En tom/ \_\_NOP while løkke kjører så lenge stopp knappen er holdt og hindrer andre inputs. Når stopp knappen slippes vil den først sjekke om den er i en gyldig etasje, og dersom den er det kaller den Utskrift 4. Hvis ikke vil tilstanden endres til stopp idle tilstanden.

## Stopp idle tilstand

Denne tilstanden er et spesialtilfelle av idle tilstanden. Den fungerer på mange måter likt, med unntak av at den avhenger av tilstanden den var i før stopp knappen ble trykket på, for å bestemme retningen den går når den får en ordre. Altså heisen vil alltid vite forrige etasjen den var i, men dersom den er mellom to etasjer så veit den for eksempel at den er mellom 2. og 1. etasje dersom forrige etasje var 2 og tilstanden dens er ned. Denne informasjonen bruker den til å fungere virtuelt på samme måte som idle tilstanden (Forrige tilstandsvariabelen vil ikke oppdateres dersom den er i stopp idle når stopp knappen trykkes). Dette tilater heisen

til å gå til etasjen den sist var på før stopp knappen ble trykket ettersom den ikke har noen sensorer som kan fortelle den hvor den er når den er mellom to etasjer.

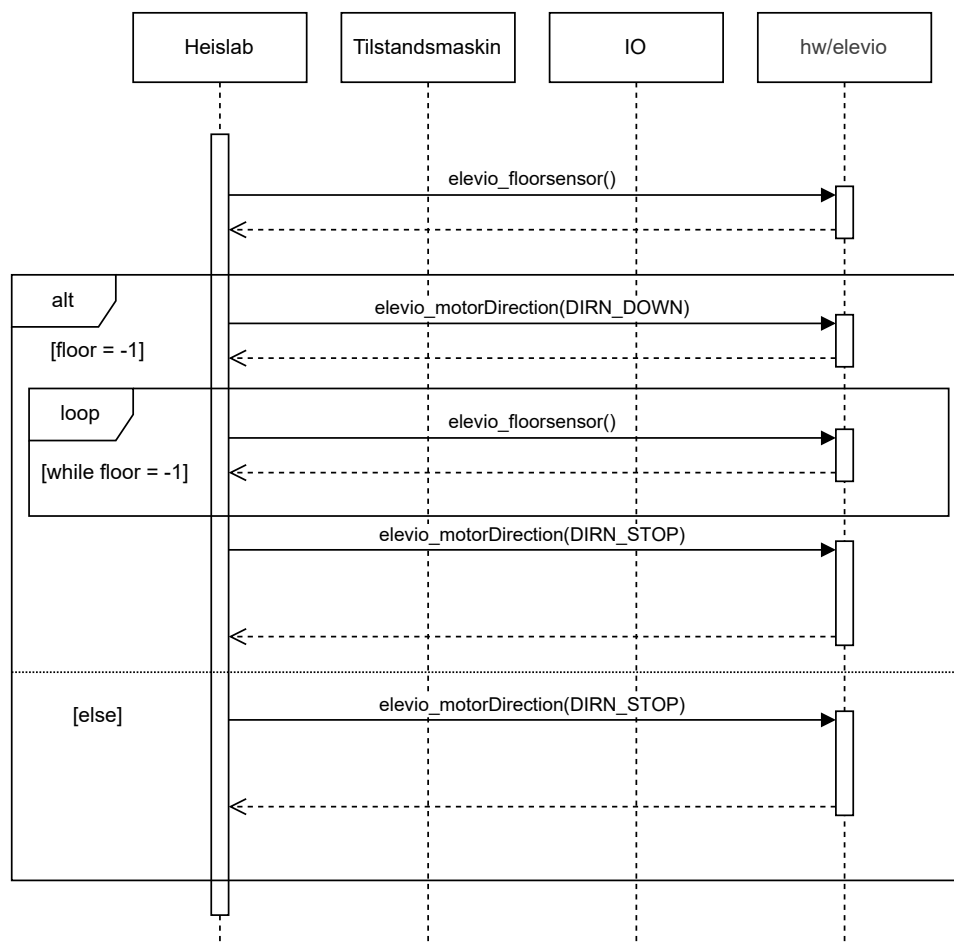
## Kodesnutter

```
typedef enum {
    MV_DWN,
    MV_IDLE,
    MV_UP,
    STOP,
    STOP_IDLE,
    INIT} ElevState;
```

Utskrift 2: Tilstands enum

```
struct ProgramState{
    ElevState elevatorState;
    ElevState previousState;
    int floor;
    int lastFloor;
    int floorLight;
    int orders[N_FLOORS][N_BUTTONS];
};
```

Utskrift 3: En struct som holder på alle variabler som beskriver heisens tilstand



Figur 5: Sekvensdiagram for initialiseringen

```

void complete_order(
    struct ProgramState* programState
);
  
```

Utskrift 4: Stopper motoren, åpner døren og fullfører ordre i nåværende etasje

```

void update_IO(
    struct ProgramState* programState
);
  
```

Utskrift 6: Oppdaterer lys og ordre utifra inputs og sensorer

```

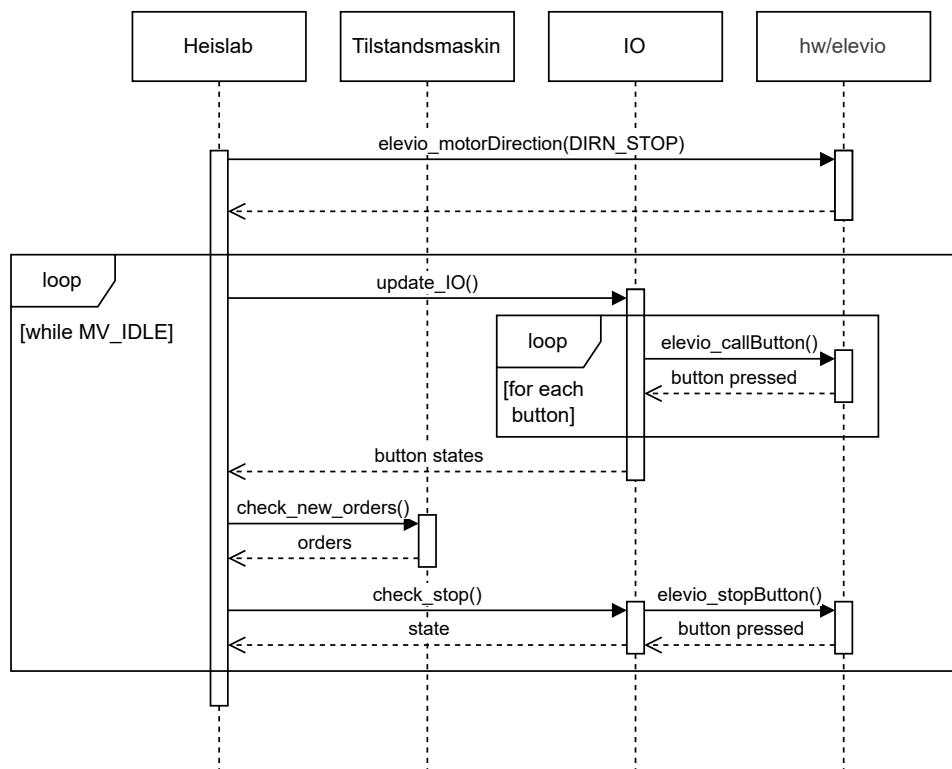
void check_orders(
    struct ProgramState* programState
);
  
```

Utskrift 5: Leser gjennom ordre og endrer tilstand til idle dersom det ikke er flere ordre i bevegelsesretning

```

void check_stop(
    struct ProgramState* programState
);
  
```

Utskrift 7: Umiddelbart endrer tilstand til stopp dersom stoppknappen blir trykket på



Figur 6: Sekvensdiagram for idle tilstanden

## Refleksjon

### V-modellen

Som nevnt i fremgangsmåten så begynte vi prosjektet med en prototype fase hvor vi testet driveren og skrev små testkodesnutter for å få erfart hva som kunne være gode indelinger av funksjonaliteter. Tilslutt ryddet vi opp og refaktorente koden etter å ha tenkt mer igjennom hvordan den burde struktureres. Vi endte hovedsakelig opp med bare en stor tilstandsmaskin istedenfor mange små moduler. Dette er fordi vi ikke fant særlig mye funksjonalitet som var fornuftig å dele opp i mindre moduler. Klassene våre i klassediagrammet er derfor også en inddeling av funksjonene våre fremfor fullt separate moduler (det er derfor tilstandsmaskinen og IO funksjonene trenger hverandre til å fungere). Den eneste faktisk separate modulen var den utdelte driveren som tok seg av all kommunikasjon med heisen. Hadde vi brukt V-modellen fra starten av istedet for å gå rett på prototyping så ville vi nok kunne finne på mange forskjellige måter å modularisere koden

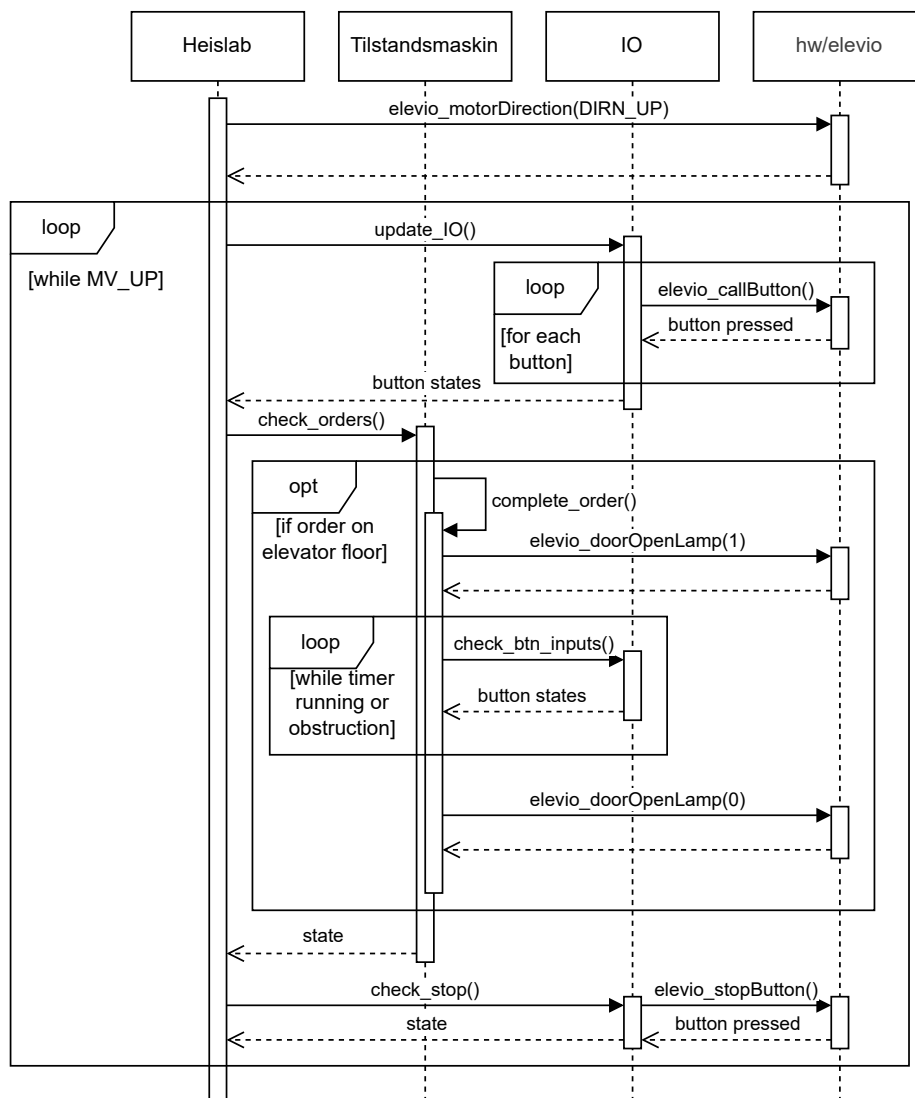
slik at mest mulig av funksjonene kan (i teorien) gjennbrukes andre steder. Derimot så har vi erfart at det ble veldig lite «boilerplate». Siden vi kun skrev det vi erfarte at vi trengte så ble koden våres knapt 252 linjer (inkludert blanke linjer). Dette er i motsetning til det vi har fått høre fra andre grupper, som har nevnt til oss at de har måtte skrive mye mer «boilerplate» for å få hver modul til å fungere helt selvstendig.

### UML

Når det gjelder bruken av UML så må det innrømmes at det ble lite brukt i starten og mye av koden ble skrevet samtidig som vi lagde de tilhørende UML diagrammene. Dette er grunnen til at vi feks har 2 versjoner/utkast av tilstandsmaskindiagrammet.

Tilstandsmaskindiagrammet ble nok det mest relevante for oss ettersom koden våres var en bonafide tilstandsmaskin. En kan nok se at andreutkastet av tilstandsmaskindiagrammet ble mer eller mindre oversatt direkte linje etter linje ifra koden. Sekvens og klassediagram derimot var ikke like nyttige for oss. Siden vi ikke hadde indelt koden i moduler fra starten av





Figur 7: Sekvensdiagram for en bestilling over heisen

så endte vi opp med å dele koden inn i de tre sudo-modulene: Heislab (main.c), Tilstandsmaskin (main.c og elevatorutils.c) og IO (elevatorutils.c). I klassediagrammet så får man kanskje inntrykket av at main.c ikke bruker noen funksjoner fra elevio direkte, men det stemmer egentlig bare «in-spirit». Dvs ikke i det hele tatt siden main.c bruker `elevio_motorDirection()` på begynnelsen av nesten alle tilstandene. Videre, siden koden ble så tett integrert med seg selv så endte også sekvensdiagrammet med å ikke være særlig nyttig eller representativt for koden våres. Feks så returnerer praktisk talt ingen av funksjonene i elevatorutils.c noe slik diagrammet sier.

Nesten alle funksjonene tar inn en peker til `programState` structen og endrer eller oppdaterer den direkte i stedet for å returnere noe. Dette gjelder både funksjonene i IO og Tilstandsmaskin «klassene».

### Skalerbarhet og robusthet

Med alt dette i bakhodet så er det nok mest riktig å beskrive koden våres som vertikalt integrert. Dette pleier ofte å være et problem med tanke på skalerbarhet siden en endring på en funksjon kan føre til endringer i helt andre deler av kodens oppførsel som på første øyekast ikke ville vært helt åpenbare. Derimot så er det veldig enkelt å utvide tilstandsmaskinen. Hver

tilstand er bare et case i en switch og for å legge til en ny tilstand så er det bare å legge til et nytt case og utvide enumen. Dermed ville vi sagt at den er skalerbar til en viss grad. Man må bare unngå å endre funksjonene som heisen allerede bruker direkte. Videre så er det verdt å nevne at selv om koden som den er nå ikke er veldig modulær, så har den tette integrasjonene gjort den grunnleggende tilstandsmaskinen ganske robust. De forskjellige tilstandene fungerer ganske uavhengig av hverandre og det er derfor lite som kan gå galt når tilstandsmaskinene blir utvidet. Samtidig er det også veldig få steder hvor det kan gå galt siden det er så lite overhead og antall funksjoner som er brukt er relativt lavt.

### **V-modellen og UMLs påvirkning på prosessen**

Videre, når det gjelder om bruken av UML og V-modellen gjorde prosessen enklere så vil vi si at det var litt av en blanding. Siden koden ikke var særlig modularisert så var det aldri noe modul-test fase, heller så skjedde mye av testingen på enten funksjonsnivå eller på systemnivå. Samtidig så ventet vi aldri med testingen av funksjoner og kode til etter implementasjonen av alt var ferdig. Det ble for det meste testet rett etter at det var skrevet, så en kan kanskje si vi fulgte en blanding av en W modell og agile modell mer enn en V-modell. Som tidligere nevnt så er det egentlig bare tilstandsmaskindigrammet som var særlig nyttig for oss, og kravet om sekvens og klassediagram gjorde at vi måtte dele koden våres inn i veldig vague grupper som ærlig talt ikke ga en god representasjon av koden. Vi ville nesten argumentert at klasse og sekvens diagrammene gir feilinntrykk av hvordan systemet fungerte i vårt tilfelle, og at inkluderingen av dem i dokumentasjon ville heller forvirret en potensiell videreutvikler av systemet enn å hjelpe. Vi fikk også inntrykket at andre grupper også har hadde lignende problemer med når de måtte lage tilstandsmaskindigrammer for kode som ikke hadde en tilstandsmaskin. Dette var i likhet med hvordan vi måtte lage klasser ut av koden når vi ikke hadde noen klasser eller moduler i

utgangspunktet. Så i det store og hele så ville vi nok sagt at deler av UML og en løsere tilnærming til V-modellen gjorde prosessen en god del enklere. Men videre så var det det stivete rammeverket og formalitetene som UML og den rene V-modellen la på prosessen som skapte mest friksjon til tider.

### **KI**

For å konkludere så er det også nødvendig å nevne bruken av KI i prosessen. Oftest når vi brukte KI så var det til å få hjelp med C syntaks siden det ofte ga oss raske og enkelte svar på ting som ellers hadde tatt et minutt eller to å lete på nettet. Det var også nyttig å få hjelp med feil som enten ikke ga noen feilmelding eller ga en veldig vag feilmelding. Dette ble mest tydelig når vi feks hadde ingen feilmelding og våres feilforståelse av hvordan pekere fungerte førte til at vi ikke misstenkte kode vi hadde skrevet med selvtillit. Derimot bør det nevnes at når vi visste hva vi ikke visste så var w3schools.com ofte bedre enn KI for å få en rask og enkel forklaring på ting. Spesielt siden det ikke var så «bloated» svar med masse ekstra info som kunne forvirre oss. Videre bør det presiseres at vi ikke brukte KI til å skrive kode direkte for oss. Det var aldri et tilfelle hvor vi spurte KI om å skrive en funksjon for oss slik at vi kunne kopiere og lime inn koden. Grunnen til det var at vi ville prinsipielt unngå å bruke KI, men hovedsakelig var det fordi at tiden det tok å forklare hva funksjonen skulle gjøre og hvordan den skulle fungere sammen og operere på det tett integrerte systemet vårt ville ofte ta mye lengre tid enn å bare skrive funksjonen selv for så å teste og feilsøke den. Alt i alt så mener vi at KI var et nyttig verktøy for å få raske svar på spørsmål vi hadde om kode vi selv hadde skrevet.

Link til github repoet vårt:

<https://github.com/Wiacer/Heislab>

Merk at det vil være commits utenom oss fra github profiler som var sist pålogget på heislabben (vi gadd/turte ikke logge inn med vår egen konto hver gang)